

Article

A Deep Learning Method of Credit Card Fraud Detection Based on Continuous-Coupled Neural Networks

Yanxi Wu ¹, Liping Wang ², Hongyu Li ³ and Jizhao Liu ^{1,4,*} ¹ School of Information Science and Engineering, Lanzhou University, Lanzhou 730000, China; wuyx22@lzu.edu.cn² Wuhan Maritime Communication Research Institute, Wuhan 430079, China; wlp722@126.com³ Henan Costar Group Co., Ltd., Nanyang 473000, China; lihongyu202801@126.com⁴ National-Local Joint Engineering Laboratory of Building Health Monitoring and Disaster Prevention Technology, Hefei 230601, China

* Correspondence: liujz@lzu.edu.cn

Abstract: With the widespread use of credit cards in online and offline transactions, credit card fraud has become a significant challenge in the financial sector. The rapid advancement of payment technologies has led to increasingly sophisticated fraud techniques, necessitating more effective detection methods. While machine learning has been extensively applied in fraud detection, the application of deep learning methods remains relatively limited. Inspired by brain-like computing, this work employs the Continuous-Coupled Neural Network (CCNN) for credit card fraud detection. Unlike traditional neural networks, the CCNN enhances the representation of complex temporal and spatial patterns through continuous neuron activation and dynamic coupling mechanisms. Using the Kaggle Credit Card Fraud Detection (CCFD) dataset, we mitigate data imbalance via the Synthetic Minority Oversampling Technique (SMOTE) and transform sample feature vectors into matrices for training. Experimental results show that our method achieves an accuracy of 0.9998, precision of 0.9996, recall of 1.0000, and an F1-score of 0.9998, surpassing traditional machine learning models, which highlight CCNN's potential to enhance the security and efficiency of fraud detection in the financial industry.



Academic Editors: Xiaobing Feng and Ioannis Tsoulos

Received: 14 January 2025

Revised: 16 February 2025

Accepted: 26 February 2025

Published: 28 February 2025

Citation: Wu, Y.; Wang, L.; Li, H.; Liu, J. A Deep Learning Method of Credit Card Fraud Detection Based on Continuous-Coupled Neural Networks. *Mathematics* **2025**, *13*, 819. <https://doi.org/10.3390/math13050819>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: CCNN; brain-like computing; credit card fraud; financial fraud; class imbalance; SMOTE

MSC: 68-02; 91-02

1. Introduction

As a major threat in the global financial industry, credit card fraud not only causes huge economic losses to consumers and businesses but also poses a serious challenge to the security and trustworthiness of the entire financial system [1,2]. According to statistics, the annual direct losses caused by credit card fraud amount to billions of dollars, while the indirect losses are even more incalculable [3]. For example, in 2023, global payment card fraud losses totaled USD 33.83 billion, reflecting a certain degree of increase from USD 33.45 billion in 2022 as 1.1% year-over-year growth. Despite intensified anti-fraud measures, fraud losses continue to rise, with e-commerce and Card-Not-Present (CNP) transactions remaining the highest-risk categories [4]. Furthermore, with the popularity of electronic payments and online transactions, credit card fraud has become more diverse and complex, which further increases the difficulty of fraud detection [5]. Therefore, how

to efficiently and accurately identify and prevent credit card fraud has become a critical issue that needs to be addressed.

Typical credit card fraud detection mainly uses machine learning techniques to distinguish between normal and fraudulent transactions by building classification models. These methods use feature information in historical transaction data to train models that can effectively detect abnormal patterns, including traditional methods such as Support Vector Machine (SVM), Random Forest (RF), Logistic Regression (LR) [5,6]. However, credit card transaction data usually suffer from a serious imbalance problem, i.e., fraudulent transactions account for only a very small percentage of all transactions, which makes traditional machine learning methods easily biased towards the majority class of samples and difficult to accurately predict fraudulent behavior in the minority class [2,7]. In addition, the high dimensionality of transaction data and the need to protect privacy pose additional challenges for model design and training. Therefore, how to extract effective features in unbalanced data to improve the robustness and generalization of fraud detection remains a focus of attention in academia and industry.

Inspired by brain-like computing, this work proposes a Continuous-Coupled Neural Network (CCNN)-based approach for credit card fraud detection. Brain-like computing is a computational model that mimics the neuronal and synaptic activities of the human brain, with asynchronous and parallel processing characteristics, and shows excellent adaptability and high efficiency in dealing with high-dimensional, nonlinear data, and complex tasks [8]. In recent years, brain-like computational methods have made significant progress in the fields of image recognition, signal processing, etc. Their unique advantages in sparse data modeling, time series analysis, and other problems provide new research ideas for credit card fraud detection. The CCNN simulates the brain's ability to perceive and process complex patterns continuously through successive convolutional operations and is able to efficiently extract potential features from transaction data. Especially in the scenario of sparse samples and hidden patterns, the CCNN shows high robustness [9].

To address the common class imbalance problem in credit card transaction data, i.e., the number of fraudulent transactions is significantly less than that of normal transactions, we adopt the synthetic minority class oversampling technique (SMOTE) to preprocess the data. The SMOTE makes the data distribution balanced by generating new minority class samples in the feature space, thus mitigating the negative impact of imbalanced data on the training of the model and improving the model's recognition of the minority class samples [10–13].

Combining the descriptions in the previous two paragraphs, the main contributions of this work are summarized as follows:

- Our work introduces brain-like computing into the field of financial fraud detection, opening up new possibilities for enhancing fraud detection systems.
- By mimicking the neural processes of the human brain, the brain-like computing method significantly improves the ability to detect credit card fraud, with an accuracy of 0.9998, precision of 0.9996, recall of 1.0000, and an F1-score of 0.9998, offering a more sophisticated approach than traditional techniques.
- The incorporation of SMOTE resampling in the training of the CCNN model effectively mitigates the challenges posed by class imbalance, which often lead to suboptimal training performance when dealing with fraudulent and normal transaction samples in the dataset.

2. Related Works

Currently, the main approaches to credit card fraud detection can be categorized into two types: machine learning methods and deep learning techniques.

2.1. Credit Card Fraud Detection Based on Machine Learning

Machine learning (ML) is an essential tool for credit card fraud detection, offering advanced methods to identify fraudulent transactions with high accuracy. Among these, Random Forest (RF) is widely used as an ensemble learning technique, combining multiple decision trees to improve classification precision. RF is particularly effective in handling large and imbalanced datasets, which are common in real-world fraud detection scenarios. However, its application to real-time data streams may be constrained by computational limitations [14]. Support Vector Machines (SVMs) have proven effective for high-dimensional classification tasks and are commonly employed in detecting fraudulent activity. While SVMs yield accurate results for smaller datasets, their efficiency diminishes when applied to large-scale data due to their computational and memory-intensive nature [15]. Similarly, K-Nearest Neighbors (KNNs) has been utilized for its simplicity and interpretability in classifying fraudulent transactions based on proximity. Despite achieving high accuracy in specific datasets, KNN's performance is hindered by its reliance on large amounts of memory and susceptibility to noisy or imbalanced data [16]. Hybrid approaches, which integrate multiple ML algorithms, offer a promising solution to these limitations. For example, models combining Random Forest and Isolation Forest have demonstrated enhanced capabilities in detecting anomaly-based fraudulent transactions while maintaining computational efficiency [17]. These methods collectively illustrate the potential of ML techniques to improve fraud detection systems, though challenges related to scalability and real-time applicability remain.

2.2. Credit Card Fraud Detection Based on Deep Learning

In recent years, deep learning models based on neural networks have become an effective fraud detection tool.

Rushin et al. [18] investigated the application of Autoencoders (AEs) in credit card fraud detection, and the results showed that deep learning models outperformed traditional methods such as gradient boosted trees and logistic regression. This suggests that deep learning methods can significantly improve the accuracy of fraud detection by learning complex patterns and relationships between data. Bhattacharyya et al. [19] presented a study on credit card fraud detection using deep data mining approaches, which demonstrated that the method they used are able to identify fraud more efficiently than traditional methods when dealing with large-scale data. This approach exploits the advantages of deep learning on large-scale datasets and is able to capture deep features in the data, thus improving the efficiency and accuracy of fraud detection. Raval et al. [20] proposed RaKShA, an explainable Long Short-Term Memory (LSTM) model for credit card fraud detection, integrating Explainable AI (XAI) with LSTM to enhance transparency and interpretability. XAI helps extract key features from the dataset, improving model performance. The X-LSTM model achieved 99.8% accuracy over 50 epochs compared to 85% for a simple LSTM. The results were stored on a blockchain to ensure tamper-proof, trusted records. This approach shows strong potential for real-world financial applications. Mienye et al. [21] proposed a hybrid deep learning framework that combines Generative Adversarial Networks (GAN) and Recurrent Neural Networks (RNN) to enhance credit card fraud detection. The framework generates realistic synthetic fraudulent transaction data via GAN to address data imbalance and enhance the quality of the training set. Then, using various deep learning architectures such as a simple RNN, LSTM, and a gated recurrent unit (GRU), the discriminator is trained to distinguish between real and synthetic transactions, and the model is further refined to identify whether a transaction is fraudulent or legitimate. Du et al. [22] proposed a hybrid method of AE-XGB-SMOTE-CGAN, combining an AE, XGBoost, the SMOTE, and a GAN. The approach addresses class imbalance by using the SMOTE for oversampling

and a GAN to generate realistic synthetic data. The method outperformed KNN and Light GBM, with a 2% improvement in accuracy and a 30% improvement in MCC, demonstrating its effectiveness in fraud detection.

Although the application of deep learning in fraud detection has achieved some results, existing deep learning methods for credit card fraud detection are still weak in the face of class imbalance and the processing of time series data.

2.3. Brain-like Computing

The human brain consists of billions of neurons, where dendrites act as the primary input interface [8]. The cell body aggregates incoming electrical signals from various dendritic branches and generates an action potential once a certain membrane threshold is exceeded. These action potentials travel through the axons and are transmitted to other neurons via synaptic connections [23,24].

Spiking neural networks (SNNs) have gained significant attention in recent years as an important model in neural computing. The research interest in SNNs continues to grow [8,25,26], with advancements such as multi-threshold augmented SNNs [27], vform [28], deep SNNs with knowledge distillation [25], and time-to-first-spike (TTFS) mechanisms in SNNs [29]. Recently, brain-like computing techniques have found applications in diverse areas like affective computing [30], mental health monitoring [31], and smart home technologies [32].

At present, various brain-like computational models are available. Wang et al. [33] proposed a Wang–Zhang (W–Z) neuron model based on neural energy theory, which is equivalent to the Hodgkin–Huxley (H–H) model through analytical dynamics. This model provides a new framework for neural information processing and effectively explains the interaction of multiple brain systems. Their neural energy theory offers unique advantages in neural modeling and information processing, advancing the integration of experimental and theoretical neuroscience. However, one of the most popular methods for the primary visual cortex is the Pulse-Coupled Neural Network (PCNN). The concept of the PCNN was introduced in the 1990s by Eckhorn and colleagues [34], who developed the model to simulate the synchronized activity of neural impulses based on their observations of synchronized oscillations in the cat’s visual cortex [34,35]. The research has consistently demonstrated that the PCNN offers notable advantages over other neural network models, especially in the realm of image classification [35].

Evolved from the PCNN, the CCNN overcomes the deterministic limitations of PCNN models, which struggle to realistically mimic the dynamic and complex behavior of human brain neurons [9]. With its ability to exhibit both periodic and chaotic behavior under varying stimuli, the CCNN has demonstrated potential in diverse fields, including image recognition and anomaly detection.

Despite its potential, the CCNN has not yet been applied to the domain of financial fraud detection, including credit card fraud. The unique characteristics of the CCNN, such as its ability to handle dynamic, chaotic patterns and adapt to changing inputs, align well with the nature of fraud detection, where fraudulent behaviors often evolve over time to bypass existing detection mechanisms.

3. Method

In this section, we propose a framework for credit card fraud detection based on the mechanisms of human brain neurons.

3.1. Data Balancing

We utilized the Kaggle CCFD dataset, which contains 284,807 transactions collected over two days. The dataset is highly imbalanced, with 284,315 normal transactions (accounting for 99.83% of the total samples) and only 492 fraudulent transactions (accounting for 0.17% of the total samples) (see Table 1). This dataset is publicly available on Kaggle and is one of the primary datasets used for credit card fraud detection research [36].

Table 1. Class distribution in the Kaggle CCFD dataset.

Class	Number of Samples	Percentage
Normal	284,315	99.83%
Fraudulent	492	0.17%
Total	284,807	

The Kaggle CCFD dataset consists of four primary features, which are detailed in Table 2. The Time feature records the timestamp of each transaction in seconds since the first transaction. The V1~V28 features are principal components obtained through Principal Component Analysis (PCA). The Amount feature indicates the monetary value of each transaction. The Class feature serves as the target variable, where a value of 1 denotes a fraudulent transaction, and 0 represents a legitimate transaction.

Table 2. Features of the Kaggle CCFD dataset.

Feature	Description
Time	Timestamp of each transaction in seconds since the first transaction.
V1~V28	Principal components from PCA to anonymized data.
Amount	Transaction amount.
Class	Label indicating fraud (1) or normal (0).

In 2002, Chawla et al. [37] proposed the SMOTE, which balances the dataset by generating synthetic minority class samples to enhance the model's ability to recognize the minority classes. The SMOTE not only effectively mitigates the class imbalance problem but also helps to reduce the overfitting and improve the model's generalization performance. This technique is widely used in various fields, which significantly improves the classification effect in unbalanced data environments [38–40].

The idea of the SOMTE algorithm is to generate new, virtual minority class samples in the feature space to balance the dataset. The specific steps include the following: first, randomly selecting a minority class sample; second, finding the nearest neighbors of that sample in the set of minority class samples; and finally, synthesizing a new minority class sample by interpolating features between the selected sample and its neighbors [37].

The specific steps of the SMOTE for the Kaggle CCFD dataset are shown in the Algorithm 1. First, minority class samples (i.e., fraudulent transactions) are extracted from the dataset. Then, for each minority class sample, its k -nearest neighbors in the feature space are computed. Next, new synthetic samples are generated by performing random interpolation between the selected samples and their neighbors. Finally, these synthetic minority class samples are combined with the original majority class samples (i.e., normal transactions) to form an oversampled balanced dataset.

Algorithm 1 SMOTE for Kaggle CCFD dataset.

Require: Dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, where $x_i = (Time, V1, \dots, V28, Amount)$ and $y_i \in \{0, 1\}$. Minority class samples $\mathcal{D}_{\text{minority}}$, majority class samples $\mathcal{D}_{\text{majority}}$. Number of synthetic samples to generate $N_{\text{synthetic}}$, number of nearest neighbors k .

Ensure: Augmented dataset $\mathcal{D}_{\text{augmented}}$

- 1: Initialize $\mathcal{D}_{\text{augmented}} \leftarrow \mathcal{D}$
- 2: Extract minority class features: $\mathcal{X}_{\text{minority}} = \{x_i \mid y_i = 1\}$
- 3: Compute k nearest neighbors for each sample $x \in \mathcal{X}_{\text{minority}}$ using Euclidean distance in the feature space
- 4: **for** $i = 1$ to $N_{\text{synthetic}}$ **do**
- 5: Randomly select a minority class sample $x_i \in \mathcal{X}_{\text{minority}}$
- 6: Randomly select one of its k -nearest neighbors x_{nn}
- 7: Generate a random interpolation factor $\lambda \sim U(0, 1)$
- 8: Compute a synthetic sample:

$$x_{\text{new}} = x_i + \lambda \cdot (x_{nn} - x_i)$$

- 9: Append x_{new} to $\mathcal{X}_{\text{minority}}$
- 10: **end for**
- 11: Combine augmented minority class samples with majority class samples:

$$\mathcal{D}_{\text{augmented}} = \{(x, 1) \mid x \in \mathcal{X}_{\text{minority}}\} \cup \mathcal{D}_{\text{majority}}$$

- 12: **return** $\mathcal{D}_{\text{augmented}}$

As is shown in Figure 1, after applying the SMOTE method, the number of fraudulent samples in the Kaggle CCFD dataset increased significantly from 492 to 284,315, achieving class balance. Meanwhile, the number of non-fraudulent samples remained unchanged at 284,315.

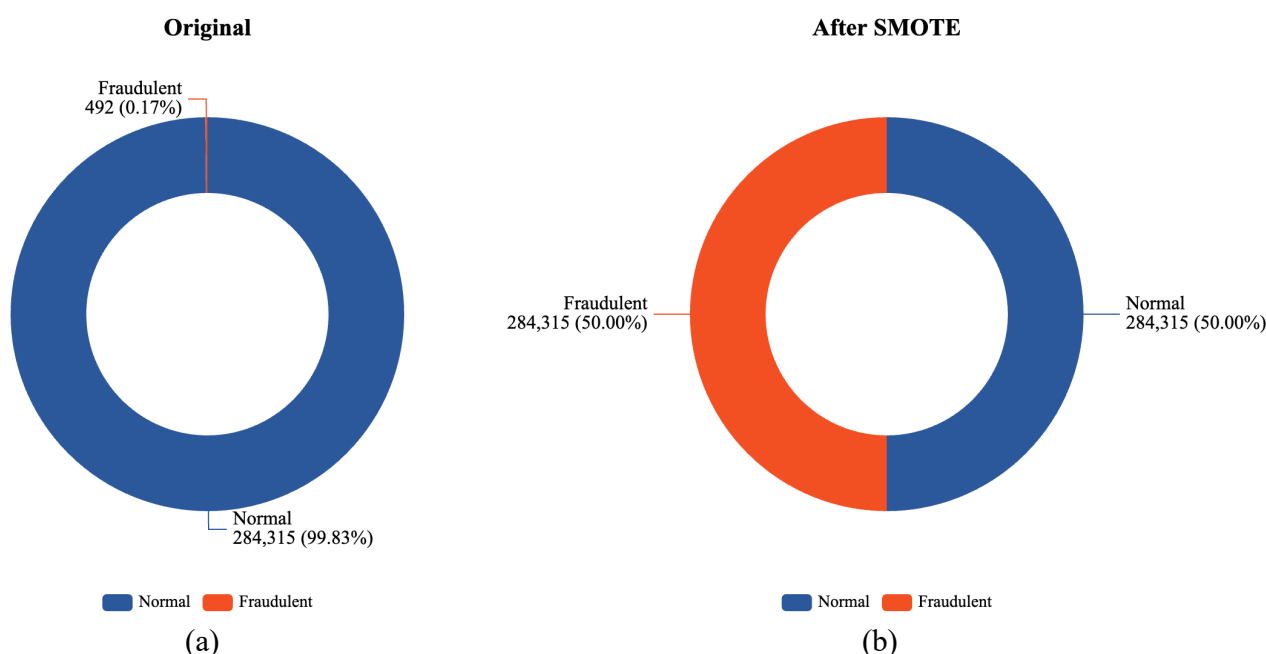


Figure 1. Comparison before and after dataset balancing. (a) original; (b) after SOMTE.

Since the dataset after resampling became very large, we used a 7:3 ratio to split the dataset into training and test datasets [41]. The number of samples after splitting is shown in the Table 3.

Table 3. Sample distribution after splitting the dataset.

Class	Training	Test
Normal	199,166	85,149
Fraudulent	198,875	85,440

3.2. Continuous-Coupled Neural Networks

The CCNN is a new neural network model derived from the PCNN. Its design goal is to more closely match the dynamics of real neurons under dynamic stimuli. Unlike the PCNN, the output of the CCNN is a continuous value, which improves the limitation of the discrete pulse output in the PCNN by introducing a probabilistic activation mechanism and thus is able to simulate the complex behaviors of biological neurons in a more comprehensive way.

The mechanism of the CCNN is described through a set of recursive equations. The state of each CCNN neuron is determined by five main modules: feeding input, couple linking, modulation product, dynamic activity, and continuous outputs [9], which are defined in Equation (1) below:

$$\begin{cases} F_{ij}(n) = e^{-\alpha_f} F_{ij}(n-1) + V_F M_{ijkl} * Y_{kl}(n-1) + S_{ij} \\ L_{ij}(n) = e^{-\alpha_l} L_{ij}(n-1) + V_L W_{ijkl} * Y_{kl}(n-1) \\ U_{ij}(n) = F_{ij}(n)(1 + \beta L_{ij}(n)) \\ Y_{ij}(n) = \text{sigmoid}(U_{ij}(n) - E_{ij}(n)) \\ E_{ij}(n) = e^{-\alpha_e} E_{ij}(n-1) + V_E Y_{ij}(n-1) \end{cases} \quad (1)$$

where the explanation of the notations is provided in Table 4.

Table 4. Notation list of CCNN neuron.

Symbol	Explanation
*	Convolution operation, representing the interaction between the weight matrix and the output signal from previous time steps.
$\alpha_f, \alpha_l, \alpha_e$	Decay factors that record previous input states.
V_F, V_L	Weighting factors modulating the action potentials of the surrounding neurons.
β	Linking strength that directly determines L_{ij} in the modulation product U_{ij} .
F_{ij}	Feeding input, reflecting the current signal received by the neuron.
S_{ij}	External feeding input received by the receptive fields.
L_{ij}	Couple linking, representing the interactions between neurons.
U_{ij}	Modulation product, determined by both the feedback input and the coupling connection input.
Y_{ij}	Continuous output, calculated by the sigmoid activation function.
E_{ij}	Dynamic activity, used to adjust the activation threshold of neurons.
M_{ijkl}, W_{ijkl}	Weight matrix, defining the strength of the connections between neurons, whose values are automatically adjusted during training.

Figure 2 shows the computational process of the CCNN neuron. The CCNN neuron updates its feeding input F_{ij} at each time step by considering the exponentially decayed previous input, the weighted sum of neighboring neuron outputs, and an external stimulus. The coupling link L_{ij} is computed similarly, capturing interactions between neurons. The modulation step combines F_{ij} and L_{ij} , amplifying or suppressing the signal through a scaling factor $1 + \beta L_{ij}$. The output Y_{ij} is obtained by applying a sigmoid activation function to the modulated signal, which is adjusted by the dynamic activity E_{ij} . The dynamic activity itself adapts over time, incorporating past output values and an exponential decay term and allowing the CCNN neuron to adjust its activation threshold dynamically.

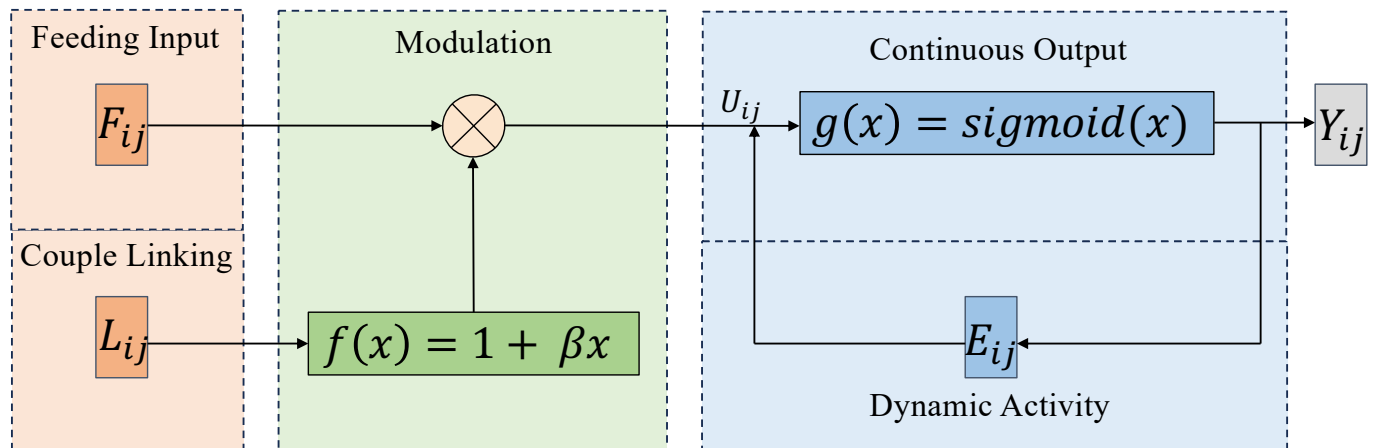


Figure 2. CCNN neuron schematic.

Based on the work of Yi et al. [42], we set the hyperparameters (α_f , α_l , α_e , β , V_E , and V_L) in the CCNN neuron as follows:

$$\begin{cases} e^{-\alpha_f} = 0.5 \\ e^{-\alpha_l} = 0.5 \\ e^{-\alpha_e} = 0.7 \\ \beta = 1.0 \\ V_E = 1.0 \\ V_L = 1.0 \end{cases} \quad (2)$$

Figure 3 shows the whole training process of the CCNN model for credit card fraud detection.

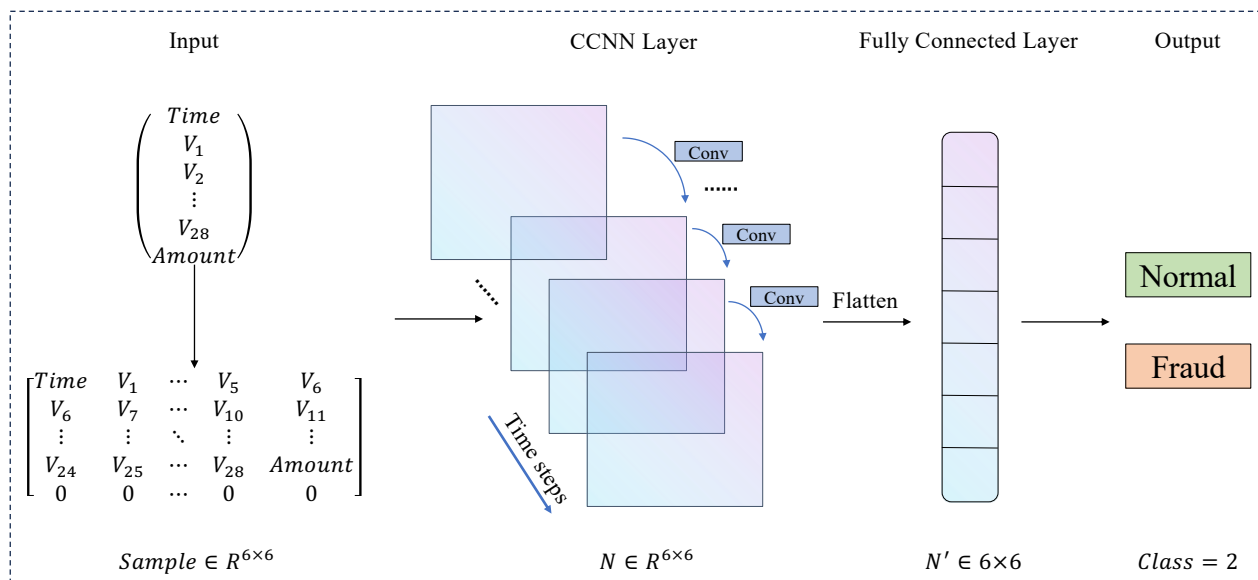


Figure 3. CCNN model for credit card fraudulent detection.

For the model input, we transform the feature vectors of the Kaggle CCFD dataset into input in the form of a matrix (see Section 4.1). Then we move into the CCNN layer, followed by spreading into the fully connected layer, and finally outputting the results.

The CCNN model in Figure 3 we designed aims to achieve classification functionality with the simplest network structure. It utilizes a CCNN neuron array that matches the size of the input samples as the CCNN layer, enabling the precise capture of sample details. The use of two connection matrices, W and M , with bias-free convolution operations allows the network to extract image features at multiple scales, thereby enhancing the model's ability to comprehend sample information.

The training process of the CCNN model we designed is shown in Algorithm 2.

Algorithm 2 CCNN model training algorithm.

- 1: **Initialize:** $F(0) = 0, L(0) = 0, U(0) = 0, E(0) = \frac{V_E}{e^{-\alpha_e}}, Y(0) = 0$
 - 2: **Initialize network parameters:** W and M
 - 3: Input X into the **CCNN Layer** to obtain y_{ccnn}
 - 4: Input y_{ccnn} into the fully connected layer to obtain y_{out}
 - 5: Compute the loss function: loss
 - 6: Compute the gradients: Autograd
 - 7: Update network parameters: W and M
-

As is shown in Algorithm 2, the training process of the CCNN model begins by initializing the states $F(0)$, $L(0)$, $U(0)$, and $Y(0)$ to zero, along with the network parameters W and M . The input x is then passed through the CCNN layer to obtain y_{ccnn} , which is further processed by the fully connected layer to produce the final output y_{out} . The loss function is computed, gradients are calculated using autograd, and the network parameters W and M are updated accordingly.

Specifically, in the initial state, all neurons are inactive, and all parameters in the state matrix are set to zero, as is shown in Equation (3) below:

$$F(0) = L(0) = Y(0) = 0, E(0) = \frac{V_E}{e^{-\alpha_e}} \quad (3)$$

The key of the CCNN consists of two dynamic connection matrices, W and M , which represent the interactions between neurons and the information transfer mechanisms. By adaptively adjusting these connections, the CCNN simulates the information processing in biological neural systems, enabling the network to autonomously extract sample features without prior knowledge.

The CCNN layer iteration process is shown in Algorithm 3.

Algorithm 3 CCNN layer iteration algorithm.

- 1: **Input:** $X = [X^1, X^2, \dots, X^T]$
 - 2: **Output:** y_{ccnn}
 - 3: Create an empty list $Y = []$
 - 4: **for** $n = 1, 2, 3, \dots, T$ **do**
 - 5: $F(n) = e^{-\alpha_f} F(n-1) + M * Y(n-1) + X(n)$
 - 6: $L(n) = e^{-\alpha_l} L(n-1) + W * Y(n-1)$
 - 7: $U(n) = F(n)(1 + \beta L(n))$
 - 8: $Y(n) = \frac{1}{1 + e^{-(U(n) - E(n))}}$
 - 9: $E(n) = e^{-\alpha_e} E(n-1) + V_E Y(n-1)$
 - 10: **end for**
 - 11: Append $Y(n)$ to the end of list Y
-

As is shown in Algorithm 3, the CCNN layer iterates over the input sequence X in time step T , updating the states $F(n)$, $L(n)$, $U(n)$, $Y(n)$, and $E(n)$ at each step based on previous values and the input data. After completing the iterations, the output of the final iteration y_{ccnn} is obtained.

After y_{cnn} is passed into the fully connected layer, the final output of the network is

$$y_{out} = A_2 \cdot (\text{ReLU}(A_1 \cdot y_{cnn} + c_1)) + c_2 \quad (4)$$

where y_{out} is the output vector after the second fully connected layer used for classification; A_1 and c_1 are the weight matrix and bias of the first fully connected layer; A_2 and c_2 are the weight matrix and bias of the second fully connected layer used for classification; and ReLU is the activation function [43], which is expressed as follows:

$$\text{ReLU}(x) = \max(x, 0) \quad (5)$$

We use the cross-entropy loss function to guide the adjustment of network parameters based on the difference between the model's predicted results and the true class labels. The cross-entropy loss function directly measures the difference between the predicted probability distribution of the model and the true probability distribution of the labels, effectively capturing the model's performance across different classes [44], as shown in Equation (6) below:

$$\text{loss} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(p_{ic}) \quad (6)$$

where N represents the number of samples, and C denotes the total number of classes; y_{ic} is an indicator function that takes the value 1 when sample i belongs to class c , and it is 0 otherwise, i.e., the one-hot encoding; p_{ic} is the predicted probability of sample i belonging to class c , which is computed as

$$p_{ic} = \arg \max_i (\hat{y}_{out}) \quad (7)$$

where $\hat{y}_{out}$ is the output predicted by the fully connected layer. Additionally, N represents the number of samples, and C represents the number of classes.

Moreover, to improve training efficiency and model performance, we adopted the Adam optimizer with the initial learning rate $\eta = 10^{-3}$. This optimizer accelerates convergence by adaptively adjusting the learning rate.

4. Experiments

In this section, we describe the process of pre-processing the data, calculating the evaluation metrics as well as analyzing the classification results.

4.1. Data Input

In visual models, the input is typically a three-channel $n \times n$ matrix, where each channel represents the red, green, and blue components of an image. However, most machine learning models require the input to be a multidimensional vector. Due to this difference, directly feeding datasets suitable for machine learning into visual models presents certain challenges. To address this issue, we propose a method in which each element of the vector is sequentially placed into the matrix, thus transforming each sample in the dataset into a single-channel "image". This allows the visual model to perform feature extraction, pooling, and classification.

The Kaggle CCFD dataset used in this study contains 30 features, which include Times, the PCA values from V1 to V28, and Amounts. These 30 features are sequentially placed into a 6×6 matrix, filling any excess space with zeros.

Specifically, denote the feature vector of the Kaggle CCFD dataset as

$$\mathbf{v} = \begin{pmatrix} Time \\ V_1 \\ V_2 \\ \vdots \\ V_{28} \\ Amount \end{pmatrix}$$

This 30-dimensional vector is transformed into a 6×6 matrix by sequentially placing the elements in a left-to-right, top-to-bottom order. Any remaining positions in the matrix are filled with zeros. The corresponding matrix representation is

$$\mathbf{M} = \begin{bmatrix} Time & V_1 & V_2 & V_3 & V_4 & V_5 \\ V_6 & V_7 & V_8 & V_9 & V_{10} & V_{11} \\ V_{12} & V_{13} & V_{14} & V_{15} & V_{16} & V_{17} \\ V_{18} & V_{19} & V_{20} & V_{21} & V_{22} & V_{23} \\ V_{24} & V_{25} & V_{26} & V_{27} & V_{28} & Amount \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

After this transformation, this matrix corresponds to a single-channel image that can be applied to a visual model. For subsequent comparison tests of machine learning models, the original input in vector form is still used.

4.2. Performance Measures

In this study, we adopted several performance metrics including accuracy, precision, recall, and F1-score to evaluate the performance of the models. These metrics provide a comprehensive assessment of the model's classification effectiveness.

We used accuracy to measure the overall predictive accuracy of the model, which represents the proportion of correctly predicted samples to the total number of samples [45]. The formula is as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (8)$$

In order to assess the model's performance specifically in predicting the positive class, we also employed precision and recall. Precision measures the proportion of predicted positive samples that are actually positive [46], and it is calculated as follows:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (9)$$

Recall focuses on the proportion of actual positive samples that are correctly predicted as positive [46], and it is calculated as follows:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (10)$$

Finally, considering the balance between precision and recall, we adopted the F1-score as a combined metric. The F1-score is the harmonic mean of the two, which helps mitigate the bias that might arise from optimizing a single metric [46]. The formula is as follows:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (11)$$

4.3. Experiment Results

We trained and tested the CCNN for 100 epochs, recording the training loss, training accuracy, testing loss, and testing accuracy for each round. The data in Figure 4 shows the performance of the model during training, with the time steps T set to 1, 2, 3, and 4, respectively.

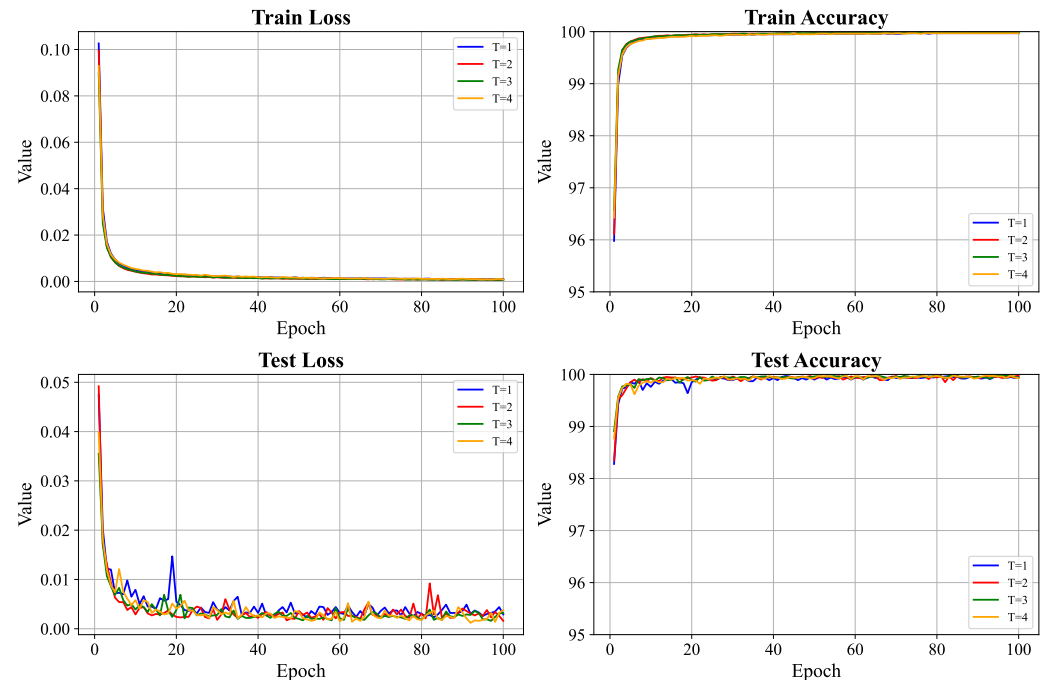


Figure 4. CCNN model training results.

From Figure 4, the training loss gradually decreased, and the training accuracy was close to 100% in almost all rounds, indicating that the model fits very well on the training dataset. The test loss also showed a decreasing trend on the whole, with occasional small fluctuations. Although the test accuracy fluctuated a little, the overall performance was still very stable.

We compared common models used for credit card fraud detection, including decision trees, logistic regression, K-Nearest Neighbors (KNNs), Light Gradient Boosting Machine (LGBM), and Naive Bayes, all of which are available directly in Python's scikit-learn (1.5.2) and lightgbm (4.6.0) library. We evaluate the models using a variety of performance metrics, including accuracy, precision, recall, and F1-scores, as described in the Section 4.2.

Table 5 shows the results of the model classification we used after training 100 epochs, and Figure 5 shows the confusion matrix.

Table 5. Classification result.

Model	TP	FP	TN	FN
Decision Tree	85,325	232	84,917	115
KNNs	85,440	185	84,964	0
LGBM	85,425	149	85,000	15
Logistic Regression	78,478	2163	82,986	6962
Naive Bayes	73,038	2129	83,020	12,402
CCNN ($T = 1$)	85,418	34	85,115	22
CCNN ($T = 2$)	85,440	35	85,114	0
CCNN ($T = 3$)	85,440	96	85,053	0
CCNN ($T = 4$)	85,440	125	85,024	0

T denotes the time step of CCNN Model.

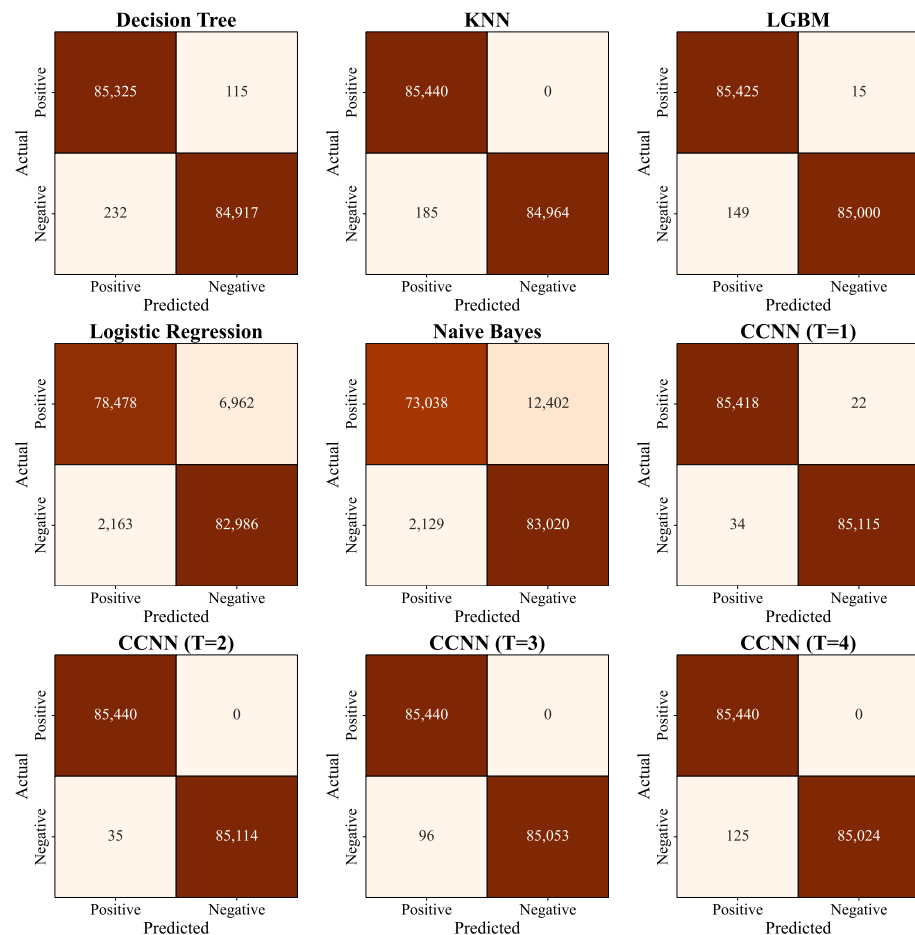


Figure 5. Confusion matrix.

Based on the results of Table 5, four metrics were calculated using the calculation methodology mentioned in the previous subsection (see Table 6). At the same time, we have included the current state-of-the-art methods in the table for comparison.

Table 6. Model performance metrics.

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree	0.9980	0.9973	0.9987	0.9980
KNNs	0.9989	0.9978	1.0000	0.9989
LGBM	0.9990	0.9983	0.9998	0.9990
Logistic Regression	0.9465	0.9732	0.9185	0.9451
Naive Bayes	0.9148	0.9717	0.8548	0.9095
CCNN ($T = 1$)	0.9997	0.9996	0.9997	0.9997
CCNN ($T = 2$)	0.9998	0.9996	1.0000	0.9998
CCNN ($T = 3$)	0.9994	0.9989	1.0000	0.9994
CCNN ($T = 4$)	0.9993	0.9985	1.0000	0.9993
XAI-SHAP [20]	99.8%	99%	98%	—
AE-XGB-SMOTE-CGAN [22]	0.9976	0.9955	0.9998	0.9976

T denotes the time step of CCNN model; the performance metrics for XAI-SHAP are taken from Table 5 of [20], which describes the results after training for 500 epochs; the performance metrics of AE-XGB-SMOTE-CGAN are calculated based on the confusion matrix data ($TP = 85,433$, $FP = 386$, $TN = 84,753$, $FN = 17$) shown in Figure 12 of [22].

From the results in the Table 6, it can be seen that the CCNN model performed exceptionally well across four metrics. When the time step was set to $T = 2$, the CCNN model achieved optimal performance in fraud-type recognition, with an accuracy of 0.9998,

indicating it correctly classifies 99.98% of the samples and demonstrating very high overall classification accuracy. The precision came out to 0.9996, meaning that 99.96% of the samples predicted as positive by the model are actually correct, showcasing excellent performance in minimizing false positives. The recall came out to 1.0000, showing that the model can identify all actual positive samples, with no false negatives, which is particularly important for high-risk tasks such as fraud detection. The F1-score came out to 0.9998, a value close to 1, indicating a good balance between precision and recall, reflecting the model's high overall performance. When the time step was set to $T \geq 3$, the model's accuracy began to decrease, but the recall remained unchanged at 1.0000. Therefore, we adopted the results with the time step set to $T = 2$ as the experimental results of the CCNN on the Kaggle CCFD dataset. In contrast, although decision tree also showed high accuracy (0.9980) and F1-score (0.9980) results, there was a slight gap in precision (0.9973) and recall (0.9987), especially in predicting positive samples, showing its deficiency in positive class identification.

The KNNs algorithm was very close to the CCNN, with an accuracy of 0.9989, precision of 0.9978, recall of 1.0000, and F1-score of 0.9989. Although its precision was slightly lower than that of the CCNN, its recall was as high as 1.0000, which suggests that KNNs is able to capture all the positive samples perfectly, but when dealing with the negative samples, KNNs does not capture all negative samples perfectly and is lower than the CCNN. The LGBM also performed well with an accuracy of 0.9990, precision of 0.9983, recall of 0.9998, and F1-score of 0.9990, showing its strong classification ability and stability, although its accuracy and precision were slightly lower than CCNN. The F1-score came out to 0.9990, showing that the LGBM has strong classification ability and stability, although it was slightly lower than the CCNN and KNNs in recall.

The performance outcomes of the logistic regression and Naive Bayes models were relatively weak, in which logistic regression performed reasonably well in accuracy (0.9465), but its precision (0.9732) and recall (0.9185) both decreased, especially in terms of recall. The performance of the Naive Bayes model was even worse: its accuracy (0.9148) and F1-score (0.9095) were lower than the former, and its recall was significantly lower (0.8548), which shows that it is weak in capturing positive samples, especially in processing unbalanced data. This shows that it is weak in capturing positive class samples.

In summary, the CCNN model we designed adopts a simplified structure yet achieves outstanding performance. The model excels in computational efficiency, requiring 32 s to train 1 epoch and 6 s for test dataset validation when running on a PC equipped with an NVIDIA® GeForce™ RTX 3070 laptop and built using PyTorch (2.6.0+cu118), with the time step set to $T = 2$. This high level of efficiency makes the model particularly suitable for real-time applications, especially in scenarios where fast response and the processing of large amounts of data are crucial, enabling real-time fraud detection.

4.4. Cross-Validation

To evaluate and verify that the model was not overfitting, we performed ten-fold cross-validation, recording the performance metrics for each fold on both the training and validation datasets. Each fold was trained for 30 epochs, which aimed to shorten the calculation time. As seen in Tables 7 and 8, the model showed very stable performance across all folds, particularly on the validation dataset, where the results were nearly perfect. The accuracy for each fold on the validation dataset was consistently high, ranging from 0.9991 to 0.9996, with recall values consistently close to 1. In particular, the recall across all folds was either 1.0000 or very close to it, indicating that the model is highly effective at identifying fraudulent samples.

Meanwhile, the precision was consistently high, varying from 0.9986 to 0.9996, with the F1-scores all above 0.9990, further indicating the model's strong performance and stability. The results across the validation dataset were almost identical to those observed on the training set (where the accuracy also consistently exceeded 0.9990), reinforcing the idea that the model is able to generalize well to unseen data without overfitting to the training dataset.

Table 7. Results of ten-fold cross-validation, with each fold trained for 30 epochs, on the training dataset.

Fold	TP	FP	TN	FN	Accuracy	Precision	Recall	F1-Score
1	255,876	193	255,623	75	0.9995	0.9992	0.9997	0.9995
2	255,623	191	255,873	80	0.9995	0.9993	0.9997	0.9995
3	255,775	205	255,711	76	0.9995	0.9992	0.9997	0.9995
4	255,926	212	255,552	77	0.9994	0.9992	0.9997	0.9994
5	255,830	188	255,685	64	0.9995	0.9993	0.9997	0.9995
6	255,797	201	255,695	74	0.9995	0.9992	0.9997	0.9995
7	255,666	174	255,853	74	0.9995	0.9993	0.9997	0.9995
8	255,903	180	255,619	65	0.9995	0.9993	0.9997	0.9995
9	255,859	193	255,646	69	0.9995	0.9992	0.9997	0.9995
10	255,845	201	255,640	81	0.9994	0.9992	0.9997	0.9994
Mean					0.9995	0.9992	0.9997	0.9995

Table 8. Results of ten-fold cross-validation, with each fold trained for 30 epochs, on the validation dataset.

Fold	TP	FP	TN	FN	Accuracy	Precision	Recall	F1-Score
1	28,359	17	28,482	5	0.9996	0.9994	0.9998	0.9996
2	28,604	26	28,225	8	0.9994	0.9991	0.9997	0.9994
3	28,464	33	28,366	0	0.9994	0.9988	1.0000	0.9994
4	28,312	32	28,519	0	0.9994	0.9989	1.0000	0.9994
5	28,421	41	28,401	0	0.9993	0.9986	1.0000	0.9993
6	28,444	30	28,389	0	0.9995	0.9989	1.0000	0.9995
7	28,575	54	28,234	0	0.9991	0.9981	1.0000	0.9991
8	28,347	21	28,495	0	0.9996	0.9993	1.0000	0.9996
9	28,454	22	28,387	0	0.9996	0.9992	1.0000	0.9996
10	28,346	10	28,464	43	0.9991	0.9996	0.9985	0.9991
Mean					0.9994	0.9990	0.9998	0.9994

This stable performance across both training and validation datasets, especially with the high accuracy, recall, and precision values, confirms that the model does not suffer from overfitting and can maintain its effectiveness even when evaluated on different data subsets.

5. Limitations and Future Work

Although our work proposes a brain-like computing-based method for financial fraud detection, achieving an accuracy of 0.9998, precision of 0.9996, recall of 1.0000, and an F1-score of 0.9998 when the time step was set to $T = 2$, there were still some limitations we found, which need to be addressed in future work:

- The SMOTE's use of interpolation to generate data may not fully capture the complex distribution of fraudulent transactions, resulting in the model fitting to synthetic data and ignoring the variability of real fraudulent transactions. We plan to find a resampling method that addresses the class imbalance problem, with performance similar to the SMOTE, but without fitting too many samples.
- For the optimal hyperparameters of the CCNN on the Kaggle CCFD dataset, a detailed grid search is required, but it is time-consuming. We plan to use a grid search with a

step size of 0.1 in our future work to find the optimal hyperparameters suitable for the Kaggle CCFD dataset.

- Our work has so far been limited to experiments on the Kaggle CCFD dataset. In future research, we plan to use a variety of mainstream credit card fraud datasets for validation. However, due to the involvement of personal privacy in transaction data and its high confidentiality, we were unable to collect such data privately and could only rely on publicly available datasets.

6. Conclusions

In this work, we propose a CCNN-based model for credit card fraud detection, which is inspired by brain-like computing. To address the class imbalance issue inherent in the dataset, we employed the SMOTE, which significantly enhances the model's ability to learn from minority class examples. The experimental results demonstrate the superior performance of our CCNN model compared to traditional machine learning models and state-of-the-art method. Specifically, our model achieved an accuracy of 0.9998, precision of 0.9996, recall of 1.0000, and an F1-score of 0.9998 when the time step was set to $T = 2$, outperforming well-established models in all key metrics such as accuracy, precision, recall, and F1-score. These results highlight the effectiveness of CCNN in handling the complexity and subtlety of fraud detection, where small, non-obvious patterns often indicate fraudulent behavior.

In conclusion, our work demonstrates that brain-like computing can significantly contribute to improving the security and efficiency of fraud detection systems in the financial sector. We designed the simplest single-layer CCNN model for training, which offers high efficiency and is well-suited for real-time detection. In the future, we plan to test this model on additional real-world datasets and explore further optimizations to enhance its generalization ability and performance in various financial fraud scenarios.

Author Contributions: Conceptualization, Y.W. and J.L.; methodology, Y.W., L.W. and H.L.; software, Y.W.; validation, Y.W., L.W. and H.L.; formal analysis, Y.W.; investigation, J.L.; resources, J.L.; data curation, L.W.; writing—original draft preparation, Y.W.; writing—review and editing, J.L.; visualization, L.W. and H.L.; supervision, J.L.; project administration, J.L.; funding acquisition, J.L. All authors have read and agreed to the published version of the manuscript.

Funding: This study is supported by the open fund project of the National-local joint Engineering Laboratory of building health monitoring and disaster prevention technology (GG23KF004) and funded in part by the Natural Science Foundation of Gansu Province under Grant 22JR5RA543.

Data Availability Statement: The datasets used for the this work are available in the Kaggle repository at <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud/data> (accessed on 30 November 2024).

Acknowledgments: Some experiments are supported by the Supercomputing Center at Lanzhou University.

Conflicts of Interest: Hongyu Li was employed by Henan Costar Group Co., Ltd. All authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Delamaire, L.; Abdou, H.; Pointon, J. Credit card fraud and detection techniques: A review. *Banks Bank Syst.* **2009**, *4*, 57–68.
2. Makki, S.; Assaghir, Z.; Taher, Y.; Haque, R.; Hacid, M.S.; Zeineddine, H. An experimental study with imbalanced classification approaches for credit card fraud detection. *IEEE Access* **2019**, *7*, 93010–93022. [CrossRef]
3. Barker, K.J.; D'amato, J.; Sheridan, P. Credit card fraud: Awareness and prevention. *J. Financ. Crime* **2008**, *15*, 398–410. [CrossRef]
4. Report, N. The Nilson Report—Issue 1276. 2024. Available online: <https://nilsonreport.com/newsletters/1276/> (accessed on 1 February 2025).

5. Popat, R.R.; Chaudhary, J. A survey on credit card fraud detection using machine learning. In Proceedings of the 2018 2nd International Conference on Trends in Electronics and Informatics (ICOEI), Tirunelveli, India, 11–12 May 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 1120–1125.
6. Varmedja, D.; Karanovic, M.; Sladojevic, S.; Arsenovic, M.; Anderla, A. Credit card fraud detection-machine learning methods. In Proceedings of the 2019 18th International Symposium INFOTEH-JAHORINA (INFOTEH), East Sarajevo, Bosnia and Herzegovina, 20–22 March 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 1–5.
7. Haixiang, G.; Yijing, L.; Shang, J.; Mingyun, G.; Yuanyue, H.; Bing, G. Learning from class-imbalanced data: Review of methods and applications. *Expert Syst. Appl.* **2017**, *73*, 220–239. [[CrossRef](#)]
8. Yi, Z.; Lian, J.; Liu, Q.; Zhu, H.; Liang, D.; Liu, J. Learning rules in spiking neural networks: A survey. *Neurocomputing* **2023**, *531*, 163–179. [[CrossRef](#)]
9. Liu, J.; Lian, J.; Sprott, J.C.; Liu, Q.; Ma, Y. The butterfly effect in primary visual cortex. *IEEE Trans. Comput.* **2022**, *71*, 2803–2815. [[CrossRef](#)]
10. Torgo, L.; Ribeiro, R.P.; Pfahringer, B.; Branco, P. Smote for regression. In Proceedings of the Portuguese Conference on Artificial Intelligence, Azores, Portugal, 9–12 September 2013; Springer: Berlin/Heidelberg, Germany, 2013; pp. 378–389.
11. Zhao, Y.; Nasrullah, Z.; Hryniewicki, M.K.; Li, Z. LSCP: Locally selective combination in parallel outlier ensembles. In Proceedings of the 2019 SIAM International Conference on Data Mining, Calgary, AB, Canada, 2–4 May 2019; SIAM: New Delhi, India, 2019; pp. 585–593.
12. Li, Z.; Zhao, Y.; Botta, N.; Ionescu, C.; Hu, X. COPOD: Copula-based outlier detection. In Proceedings of the 2020 IEEE INTERNATIONAL conference on Data Mining (ICDM), Sorrento, Italy, 17–20 November 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1118–1123.
13. Fernández, A.; Garcia, S.; Herrera, F.; Chawla, N.V. SMOTE for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary. *J. Artif. Intell. Res.* **2018**, *61*, 863–905. [[CrossRef](#)]
14. Xuan, S.; Liu, G.; Li, Z.; Zheng, L.; Wang, S.; Jiang, C. Random forest for credit card fraud detection. In Proceedings of the 2018 IEEE 15th International Conference on Networking, Sensing and Control (ICNSC), Zhuhai, China, 27–29 March 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 1–6.
15. Puh, M.; Brkić, L. Detecting credit card fraud using selected machine learning algorithms. In Proceedings of the 2019 42nd International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), Opatija, Croatia, 20–24 May 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 1250–1255.
16. Itoo, F.; Meenakshi, S.; Singh, S. Comparison and analysis of logistic regression, Naive Bayes and KNN machine learning algorithms for credit card fraud detection. *Int. J. Inf. Technol.* **2021**, *13*, 1503–1511. [[CrossRef](#)]
17. Vynokurova, O.; Peleshko, D.; Bondarenko, O.; Ilyasov, V.; Serzhantov, V.; Peleshko, M. Hybrid machine learning system for solving fraud detection tasks. In Proceedings of the 2020 IEEE Third International Conference on Data Stream Mining & Processing (DSMP), Lviv, Ukraine, 21–25 August 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1–5.
18. Rushin, G.; Stancil, C.; Sun, M.; Adams, S.; Beling, P. Horse race analysis in credit card fraud—Deep learning, logistic regression, and Gradient Boosted Tree. In Proceedings of the 2017 Systems and Information Engineering Design Symposium (SIEDS), Charlottesville, VA, USA, 28 April 2017; IEEE: Piscataway, NJ, USA, 2017; pp. 117–121.
19. Bhattacharyya, S.; Jha, S.; Tharakunnel, K.; Westland, J.C. Data mining for credit card fraud: A comparative study. *Decis. Support Syst.* **2011**, *50*, 602–613. [[CrossRef](#)]
20. Raval, J.; Bhattacharya, P.; Jadav, N.K.; Tanwar, S.; Sharma, G.; Bokoro, P.N.; Elmorsy, M.; Tolba, A.; Raboaca, M.S. Raksha: A trusted explainable lstm model to classify fraud patterns on credit card transactions. *Mathematics* **2023**, *11*, 1901. [[CrossRef](#)]
21. Mienye, I.D.; Swart, T.G. A Hybrid Deep Learning Approach with Generative Adversarial Network for Credit Card Fraud Detection. *Technologies* **2024**, *12*, 186. [[CrossRef](#)]
22. Du, H.; Lv, L.; Wang, H.; Guo, A. A novel method for detecting credit card fraud problems. *PLoS ONE* **2024**, *19*, e0294537. [[CrossRef](#)] [[PubMed](#)]
23. Ma, T.; Mou, J.; Al-Barakati, A.A.; Jahanshahi, H.; Miao, M. Hidden dynamics of memristor-coupled neurons with multi-stability and multi-transient hyperchaotic behavior. *Phys. Scr.* **2023**, *98*, 105202. [[CrossRef](#)]
24. Zheng, Y.; Yu, Z.; Wang, S.; Huang, T. Spike-based motion estimation for object tracking through bio-inspired unsupervised learning. *IEEE Trans. Image Process.* **2022**, *32*, 335–349. [[CrossRef](#)]
25. Xu, Q.; Li, Y.; Shen, J.; Liu, J.K.; Tang, H.; Pan, G. Constructing deep spiking neural networks from artificial neural networks with knowledge distillation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Vancouver, BC, Canada, 17–24 June 2023; pp. 7886–7895.
26. Li, Y.; Fang, X.; Gao, Y.; Zhou, D.; Shen, J.; Liu, J.K.; Pan, G.; Xu, Q. Efficient Structure Slimming for Spiking Neural Networks. *IEEE Trans. Artif. Intell.* **2024**, *5*, 3823–3831. [[CrossRef](#)]
27. Wang, X.; Zhang, Y.; Zhang, Y. MT-SNN: Enhance spiking neural network with multiple thresholds. *arXiv* **2023**, arXiv:2303.11127.

28. Huang, T.; Zheng, Y.; Yu, Z.; Chen, R.; Li, Y.; Xiong, R.; Ma, L.; Zhao, J.; Dong, S.; Zhu, L.; et al. $1000\times$ faster camera and machine vision with ordinary devices. *Engineering* **2023**, *25*, 110–119. [CrossRef]
29. Malcolm, K.; Casco-Rodriguez, J. A comprehensive review of spiking neural networks: Interpretation, optimization, efficiency, and best practices. *arXiv* **2023**, arXiv:2303.10780.
30. Dong, Z.; Ji, X.; Lai, C.S.; Qi, D. Design and implementation of a flexible neuromorphic computing system for affective communication via memristive circuits. *IEEE Commun. Mag.* **2022**, *61*, 74–80. [CrossRef]
31. Ji, X.; Dong, Z.; Han, Y.; Lai, C.S.; Zhou, G.; Qi, D. EMSN: An energy-efficient memristive sequencer network for human emotion classification in mental health monitoring. *IEEE Trans. Consum. Electron.* **2023**, *69*, 1005–1016. [CrossRef]
32. Dong, Z.; Ji, X.; Zhou, G.; Gao, M.; Qi, D. Multimodal neuromorphic sensory-processing system with memristor circuits for smart home applications. *IEEE Trans. Ind. Appl.* **2022**, *59*, 47–58. [CrossRef]
33. Wang, R.; Wang, Y.; Xu, X.; Li, Y.; Pan, X. Brain works principle followed by neural information processing: A review of novel brain theory. *Artif. Intell. Rev.* **2023**, *56*, 285–350. [CrossRef]
34. Eckhorn, R.; Reitbock, H.; Arndt, M.; Dicke, P. A neural network for feature linking via synchronous activity: Results from cat visual cortex and from simulations. In *Models of Brain Function*; Cambridge University Press: Cambridge, UK, 1989.
35. Liu, H.; Liu, M.; Li, D.; Zheng, W.; Yin, L.; Wang, R. Recent advances in pulse-coupled neural networks with applications in image processing. *Electronics* **2022**, *11*, 3264. [CrossRef]
36. Machine Learning Group-ULB. Credit Card Fraud Detection Dataset. 2018. Available online: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud/data> (accessed on 30 November 2024).
37. Chawla, N.V.; Bowyer, K.W.; Hall, L.O.; Kegelmeyer, W.P. SMOTE: Synthetic minority over-sampling technique. *J. Artif. Intell. Res.* **2002**, *16*, 321–357. [CrossRef]
38. Bajer, D.; Zonć, B.; Dudjak, M.; Martinović, G. Performance analysis of SMOTE-based oversampling techniques when dealing with data imbalance. In Proceedings of the 2019 International Conference on Systems, Signals and Image Processing (IWSSIP), Osijek, Croatia, 5–7 June 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 265–271.
39. Zhao, Y.; Rossi, R.; Akoglu, L. Automatic unsupervised outlier model selection. *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 4489–4502.
40. Elreedy, D.; Atiya, A.F. A comprehensive analysis of synthetic minority oversampling technique (SMOTE) for handling class imbalance. *Inf. Sci.* **2019**, *505*, 32–64. [CrossRef]
41. Bishop, C.M.; Nasrabadi, N.M. *Pattern Recognition and Machine Learning*; Springer: Berlin/Heidelberg, Germany, 2006; Volume 4.
42. Yi, Z.; Lian, J.; Qi, Y.; Yu, Z.; Tang, H.; Ma, Y.; Liu, J. Deep Pulse-Coupled Neural Networks. *arXiv* **2023**, arXiv:2401.08649.
43. Schmidt-Hieber, J. Nonparametric regression using deep neural networks with ReLU activation function. *Ann. Statist.* **2020**, *48*, 1875–1897.
44. Mao, A.; Mohri, M.; Zhong, Y. Cross-entropy loss functions: Theoretical analysis and applications. In Proceedings of the International Conference on Machine Learning, Honolulu, HI, USA 23–29 July 2023; PMLR: New York, NY, USA, 2023; pp. 23803–23828.
45. Hossin, M.; Sulaiman, M.N. A review on evaluation metrics for data classification evaluations. *Int. J. Data Min. Knowl. Manag. Process* **2015**, *5*, 1.
46. Yacoub, R.; Axman, D. Probabilistic extension of precision, recall, and f1 score for more thorough evaluation of classification models. In Proceedings of the First Workshop on Evaluation and Comparison of NLP Systems, Online, 16 March 2020; pp. 79–91.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.